

In each round, the block of data is divided into two halves. One half is encrypted using a substitution box (S-box), a permutation box (P-box), and a sub-key. An XOR operation is then performed between the encrypted half and the other half, followed by swapping the two halves. This process is repeated for multiple rounds, ultimately producing the encrypted block of data. The Feistel network's unique structure allows for efficient encryption and decryption, making it an ideal framework for block ciphers like DES.

DES offers several modes of operation, each chosen based on specific security and performance requirements. The ECB (Electronic Codebook) mode encrypts each block independently. Though easy to implement, this mode is open to pattern attacks because identical plaintext blocks yield identical ciphertext. The CBC (Cipher Block Chaining) mode enhances security by performing an XOR operation on each plaintext block with the preceding ciphertext block before encryption, thereby obscuring patterns. The CFB (Cipher Feedback) mode and OFB (Output Feedback) mode effectively transform DES into a stream cipher (which we will discuss later), allowing encryption of smaller data units and providing resilience against bit-level manipulation. The CTR (Counter) mode leverages a counter value to generate keystream blocks, enabling parallel encryption and decryption for high-throughput applications. The choice of a particular mode depends on the trade-offs between security, efficiency, and error propagation.

Now that we have a clearer understanding of how DES operates, let us explore additional implementation options. In the previous article, we used the library 'PyCryptodome' to implement DES. However, there are multiple ways to implement cryptographic functions in SageMath. Recall that SageMath is essentially Python with added mathematical capabilities, which means any package or library accessible in Python can also be used in SageMath. Moreover, with Cython and Jython, it is possible to integrate C, C++, and Java code within Python. As a result, libraries written in these languages can also be leveraged within SageMath, providing greater flexibility in cryptographic implementations.

So, what other options do we have for implementing cryptographic functions? The aptly named library 'Cryptography' is designed to be both secure and user-friendly. It includes symmetric encryption algorithms like DES and AES, key derivation functions such as PBKDF2 and Argon2, as well as support for certificates and public-key cryptography. The 'hashlib' library, which is part of Python's standard library, is used for hashing and supports algorithms like MD5, SHA-1, SHA-256, and SHA-512. Another option is 'PyNaCl', a Python binding to the 'NaCl (libsodium)' library, which focuses on secure and high-performance cryptographic operations. It provides authenticated encryption, digital signatures, and hashing. If terms like key derivation, certificates, and hashing are unfamiliar to you, don't worry—we will explore these

concepts in detail in the upcoming articles in this series.

SageMath can also access C libraries like 'OpenSSL', one of the most widely used cryptographic libraries, providing functions for SSL/TLS, encryption, decryption, and hashing. Another modern and user-friendly C library for cryptographic operations, 'libsodium', can also be utilised within SageMath. For C++-based cryptographic libraries, 'Crypto++', which offers a comprehensive suite of cryptographic algorithms, can be integrated into SageMath by creating Python bindings using 'Cython'. Similarly, Java libraries such as 'Bouncy Castle' and the 'Java Cryptography Architecture (JCA)' can be accessed in SageMath through Jython, enabling the use of Java-based cryptographic functions.

Now, it is time to discuss some history. We have already seen that the US government adopted DES as its official standard in 1977. During the Cold War, the Soviet Union (now largely succeeded by Russia) developed its own cryptographic algorithm called GOST.

What about our country? Cryptography is closely tied to national security, and information regarding it is often difficult to find in the public domain. However, the limited available information suggests that India used the Indian Naval Cipher until the 1960s and later switched to DES in the 1970s. By the 1980s, we had started developing our own indigenous cryptographic systems.

The following story highlights the importance of developing our own cryptographic standards rather than blindly adopting those created by others. In the 1970s, when DES was designed, the concept of differential cryptanalysis (a type of attack on cryptographic algorithms) was not publicly known. However, the designers of DES—IBM and the NSA (National Security Agency of the US)—were aware of this potential vulnerability. To mitigate the risk, they modified the original S-box design to make it more resistant to differential cryptanalysis.

It wasn't until the 1990s that Eli Biham and Adi Shamir independently discovered differential cryptanalysis and applied it to DES. They demonstrated that DES was indeed vulnerable to this attack, although exploiting it required an impractically large number of chosen plaintexts. When the NSA later announced that they had known about such attacks as early as the 1970s, it came as a surprise to many.

This raises an important question: Are we truly secure, or is there an agency secretly accessing our data by exploiting vulnerabilities that are not yet publicly known? Hopefully not. This concern has been widely discussed, even in fiction. For example, the novel 'Digital Fortress' by Dan Brown explores the world of encryption as its central theme.

The vulnerability of the S-box was not the primary threat to DES; rather, it was the relatively short key length of 64 bits, which became increasingly susceptible to attacks as faster computers were developed by the late 1970s. Triple DES (also known as 3DES) was introduced as an enhancement to